**24CS2506 – Building Applications using GPT-4**
**Experiment 3 - Intelligent Email Assistant using Ollama**

## 1. What is an Email Assistant?

An Email Assistant is an AI application that helps users read emails, understand customer queries, and automatically generate professional replies. Instead of writing responses manually, the AI analyzes the email and suggests an appropriate reply.

**Examples**

Customer Email
My order has not been delivered yet.
AI Reply
We apologize for the delay. Our team is checking your shipment. You will receive an update shortly.

---

## 2. What is AI Reply Generation?

AI Reply Generation is the process of generating suitable responses using Large Language Models.

The AI understands

- Customer intention
- Email tone
- Context
- Previous conversations

and generates a professional response.

---

## 3. Why use Previous Interactions?

Instead of replying randomly, AI looks at similar previous emails.

Example

Past Email
"My package arrived damaged."
Previous Reply
"We are sorry for the inconvenience. We will arrange a replacement immediately."
Current Email
"My laptop screen is broken."
Suggested Reply
"We apologize for the inconvenience. We will arrange a replacement after verification."

---

| Ex. No.:3<br>11-09-2026 | Intelligent Email Assistant using Ollama | URK23CS1105 |
|---|---|---|

**Aim:**

To design and implement an intelligent Email Assistant using Ollama that drafts professional, context-aware replies to incoming emails by combining a stored FAQ knowledge base with a local language model and conversational memory.

**Problem Statement:**

Design and develop an email assistant that identifies common repetitive email queries and answers them instantly from a pre-stored FAQ knowledge base (pricing, refund, support). Queries that do not match any FAQ keyword must be routed to a local Ollama language model (qwen2.5:0.5b), which drafts a polite and professional reply. The assistant must maintain conversational history and inject it into the prompt so that follow-up emails receive consistent, context-aware responses.

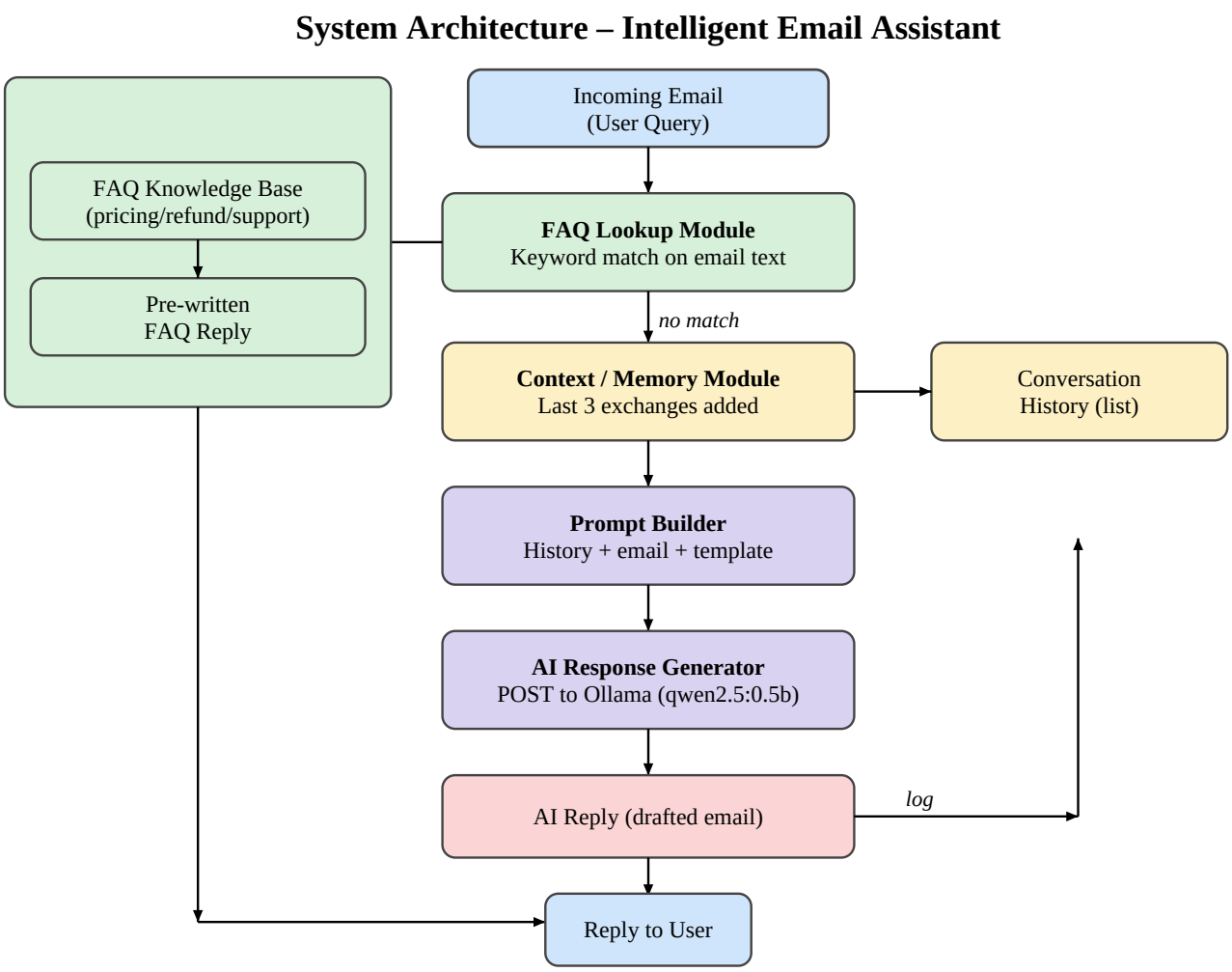
**Software Requirements:**

| Software | Version |
|---|---|
| Python | 3.14.2 |
| Ollama | v0.34.0 |
| VS Code | 1.136.1 |
| Requests | 2.x |
| Model (via Ollama) | qwen2.5:0.5b |

**Knowledge Base Description**

| Component | Purpose |
|---|---|
| FAQ Dictionary | Contains keywords (pricing, refund, support) mapped to pre-written standardized answers. Used for instant keyword-based replies to common queries. |
| Conversation History | A list storing previous incoming emails and assistant replies. The last three exchanges are injected into the AI prompt to maintain multi-turn context. |
| Ollama Model | Local qwen2.5:0.5b model served over HTTP (/api/generate). Used to draft context-aware replies for queries not covered by the FAQ. |

**System Architecture**




**Module Design**

| Module | Responsibility |
|---|---|
| FAQ Module | Scans the incoming email for FAQ keywords and immediately returns the matching pre-written reply. |
| Memory Module | Stores each email and reply in a history list and builds the context string from the last three exchanges. |
| AI Response Module | Builds the prompt template and sends it to the Ollama API via a POST request to generate a professional reply. |
| Email Assistant | Integrates all modules: checks the FAQ first, routes unmatched emails to the AI with context, and logs every exchange. |

**Implementation**

YouTube Link: https://youtu.be/CWGW4nZVBc8

**Program:**

```
import requests

# Ollama API settings
OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL_NAME = "qwen2.5:0.5b"

# Step 1: FAQ Knowledge Base
faq = {
    "pricing": "Our basic plan starts at $10/month with premium options.",
    "refund": (
        "Refunds are available within 30 days of purchase. Please share your"
        " order ID."
    ),
    "support": "You can reach support at support@example.com, 24/7.",
}

# Step 6: Maintain Context (Memory)
conversation_history = []

# Step 2: Check FAQ
def check_faq(email_text):
    for keyword, answer in faq.items():
        if keyword in email_text.lower():
            return answer
    return None

# Step 3: Generate AI Response with Ollama API
def ollama_response(prompt):
    payload = {
        "model": MODEL_NAME,
        "prompt": prompt,
        "stream": False,  # disable streaming for simplicity
    }
    try:
        response = requests.post(OLLAMA_URL, json=payload, timeout=60)
        if response.status_code == 200:
            return response.json().get("response", "").strip()
        else:
            return f"Error: {response.text}"
    except Exception as e:
        return f"Connection Error: {e}"

# Step 4, 6 & 7: Integrated Email Assistant Logic with Context / Memory
def email_assistant(email_text):
    # 1. Check FAQ first
    faq_answer = check_faq(email_text)
    if faq_answer:
        reply = "FAQ Reply: " + faq_answer
        conversation_history.append({"email": email_text, "reply": reply})
        return reply

    # 2. Append prior context if available
    context_str = ""
    if conversation_history:
        context_str = "Previous conversation history:\n"
        for item in conversation_history[-3:]:
            context_str += (
                f"- User: {item['email']}\n- Assistant: {item['reply']}\n"
            )
        context_str += "\n"
    prompt = (
```

```
            f"{context_str}"
            f"Draft a polite, professional email reply to this message:\n\n'{email_text}'"
        )

        # 3. Generate response using AI model and update memory
        ai_answer = ollama_response(prompt)
        reply = "AI Reply: " + ai_answer
        conversation_history.append({"email": email_text, "reply": reply})
        return reply

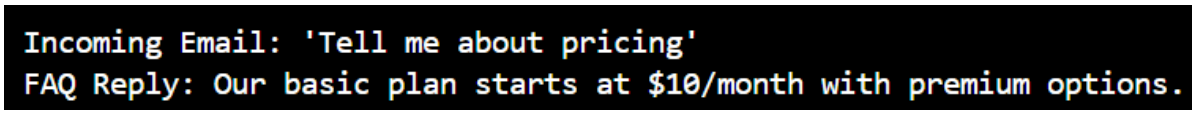
    def main():
        print("=" * 60)
        print("   Ex.3 Intelligent Email Assistant Execution")
        print("=" * 60)

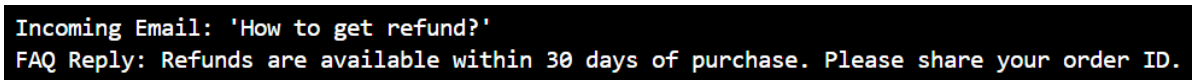
        # Step 8: Test with sample emails from manual
        emails = [
            "Tell me about pricing",
            "How to get refund?",
            "Need support help",
            "Delay in my order",
        ]
        for mail in emails:
            print(f"\nIncoming Email: '{mail}'")
            response = email_assistant(mail)
            print(response)
            print("-" * 50)

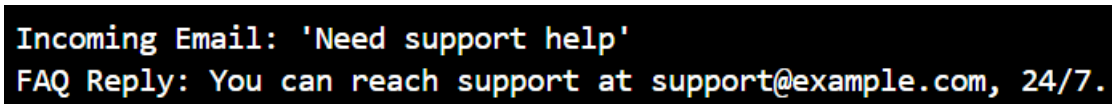
        # Test memory / context consistency
        follow_up = "Could you also expedite the delivery for that delayed order?"
        print(f"\nIncoming Follow-up Email: '{follow_up}'")
        response = email_assistant(follow_up)
        print(response)
        print("-" * 50)

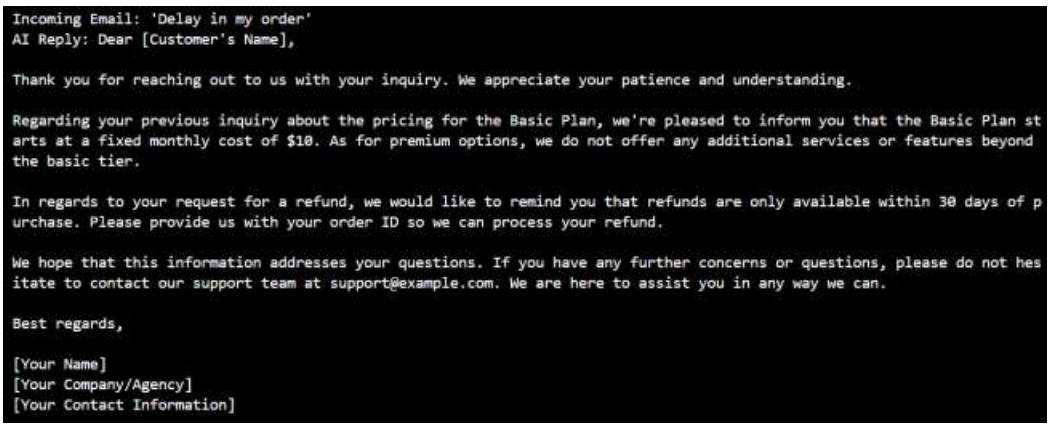
    if __name__ == "__main__":
        main()
```

**Output Screenshots:**







**Result:**

Thus successfully built a local intelligent email assistant that answers common queries instantly from an FAQ knowledge base and drafts professional, context-aware replies for complex inquiries using Ollama with conversation memory.